Language Models From Scratch

Deep Learning course notes Sources: listed under References

What this file is really about:

$$\text{tokens} \rightarrow \text{a GPT in one file} \rightarrow \text{pretraining} \rightarrow \text{fine-tuning, LoRA and RLHF} \rightarrow \text{retrieval} \rightarrow \text{ev}$$

How to use this file. Part A covers every stage in the life of a language model. Part B is a complete, runnable GPT and the surrounding tools. Part C is practice with answers.

Step-up path

You need. Attention and PyTorch basics.

1. Build a tokeniser.
2. Assemble a complete GPT.
3. Train and sample from it.
4. Adapt a model with LoRA.
5. Answer from documents and evaluate it.

You can stop when you can explain and implement each part of a language model product.

Contents

Part A Maths From the Ground Up

A1 Tokenisation

A language model reads integer tokens. Characters give a tiny vocabulary and long sequences. Words give a huge vocabulary and many unknown words. **Subword** tokenisers sit between the two.

Byte-pair encoding builds one from data:

1. Start with the 256 possible bytes.
2. Count every adjacent pair in the text.
3. Replace the most frequent pair with a new token.
4. Repeat until the vocabulary is large enough.

Common words end up as single tokens, and rare words split into pieces.

> Careful. Tokens are not words. Models see "strawberry" as a few chunks, which is one reason they miscount letters. Odd-looking failures with spelling or arithmetic often come from tokenisation.

A2 The input to a GPT

A batch has shape $B \times T$. Token embeddings and learned position embeddings, each $B \times T \times d$, are added together. Targets are the same sequence shifted left by one token: at every position the model predicts the next token.

A3 The masked attention block

Each block computes

$$x \leftarrow x + \mathrm{MaskedMultiHeadAttention}(\mathrm{LayerNorm}(x)), \qquad x \leftarrow x + \mathrm{MLP}(\mathrm{LayerNorm}(x)).$$

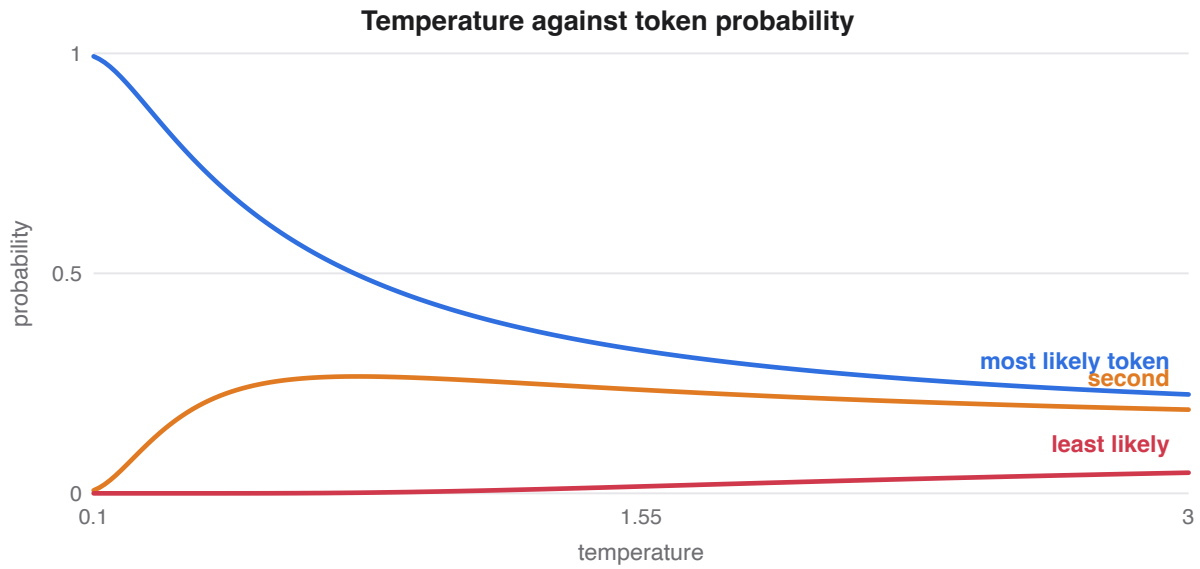
Layer normalisation normalises across features rather than across the batch, so it works for any batch size and sequence length. Residual connections keep gradients flowing through many layers.

A final linear layer maps d back to the vocabulary size, usually sharing its weights with the token embedding.

A4 Counting parameters and cost

For an attention head, the query, key and value maps cost $3d^2/h$ weights each head, so about $3d^2$ per layer in total. The MLP, of width $4d$, adds about $8d^2$. A block then has about $11d^2$ weights. With $d = 384$ and 6 layers that is around 10 million.

Attention takes $O(T^2 d)$ per layer, so long contexts are expensive.



A5 Pretraining

Pretraining minimises next-token cross-entropy over huge amounts of text. A random model has a loss of about $\ln V$, where V is the vocabulary size. Good practice includes learning-rate warmup then cosine decay, gradient clipping, and regular held-out evaluation.

A6 Fine-tuning and transfer

Fine-tuning continues training a pretrained model on your task. Freeze most of the model and train the last layers, or train everything with a small learning rate.

LoRA avoids copying all the weights. For a weight matrix W, it trains only a low-rank update:

$$W' = W + \frac{\alpha}{r} BA, \qquad B \in \mathbb{R}^{m \times r},\ A \in \mathbb{R}^{r \times n}.$$

For $m = n = 1000$ and $r = 8$ this trains 16,000 numbers rather than 1,000,000.

A7 From a text predictor to an assistant

A pretrained model continues text, and it does not naturally follow instructions.

- **Instruction tuning** trains on prompt and answer pairs.
- **RLHF** collects human preferences between pairs of answers, trains a reward model on them, and then improves the policy against that reward.
- **DPO** gets a similar result directly, with a classification-style loss on preference pairs.

Be aware of what these do not do. They change what the model prefers to say, and do not by themselves make its facts correct.

A8 Prompting

A prompt is the model's whole world. System instructions, examples and retrieved context are all just tokens. Few-shot examples work because they condition the next-token distribution. **Chain of thought** gives the model tokens to work in before answering.

A9 Retrieval-augmented generation

Models have no access to your documents, and may invent answers. A retrieval pipeline does this:

1. **Split** documents into passages.
2. **Embed** each passage into a vector.
3. **Store** the vectors in an index.
4. **Retrieve** the top-k passages for a question.
5. **Generate** an answer from them, citing what was retrieved.

Retrieval quality usually matters more than the choice of model. Evaluate it separately, with recall at k.

A10 Evaluating language models

Average accuracy hides a lot. Use a fixed held-out set of tasks with explicit pass criteria. Check that answers are grounded in retrieved text, and measure refusal rates, cost and latency. Run the suite on every change, as you would run software tests.

> **Careful.** Generated text varies from run to run. Evaluate several samples, or fix the seed.

A11 Serving

Generating one token at a time is slow without a **key-value cache**: recomputing all earlier tokens at every step makes generation quadratic, and caching each layer's keys and values makes it linear. **Quantisation** shrinks weights to 8 or 4 bits. **Batching** many requests raises throughput. **Speculative decoding** uses a small model to draft and a large model to verify.

A12 Safety in practice

Treat retrieved text and tool output as data, and never as instructions. Give agents the fewest permissions they need. Require confirmation before actions that are hard to undo.

Part B In Code

B1 Byte-pair encoding

```python
from collections import Counter

def pairs(ids):
    return Counter(zip(ids, ids[1:]))

def merge(ids, pair, new):
    out, i = [], 0
    while i < len(ids):
        if i + 1 < len(ids) and (ids[i], ids[i + 1]) == pair:
            out.append(new); i += 2
        else:
            out.append(ids[i]); i += 1
    return out

def train_bpe(text, vocab_size):
    ids = list(text.encode('utf-8'))
    merges = {}
    for new in range(256, vocab_size):
        counts = pairs(ids)
        if not counts:
            break
        best = counts.most_common(1)[0][0]
        ids = merge(ids, best, new)
        merges[best] = new
    return merges

print(train_bpe('low low low lower lowest', 262))
```

B2 A complete GPT

This single program builds, trains and samples from a character-level GPT. Put any text in `input.txt`.

```python
import math
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
device = 'cuda' if torch.cuda.is_available() else 'cpu'
block_size, batch_size, d, n_head, n_layer = 128, 64, 192, 6, 4

text = open('input.txt', encoding='utf-8').read()
chars = sorted(set(text))
stoi = {c: i for i, c in enumerate(chars)}
itos = {i: c for c, i in stoi.items()}
V = len(chars)
data = torch.tensor([stoi[c] for c in text], dtype=torch.long)
cut = int(0.9 * len(data))
train_data, val_data = data[:cut], data[cut:]


def get_batch(split):
    src = train_data if split == 'train' else val_data
    ix = torch.randint(len(src) - block_size - 1, (batch_size,))
    x = torch.stack([src[i:i + block_size] for i in ix])
    y = torch.stack([src[i + 1:i + block_size + 1] for i in ix])
    return x.to(device), y.to(device)


class SelfAttention(nn.Module):
    def __init__(self):
        super().__init__()
        self.n_head, self.d = n_head, d
        self.qkv = nn.Linear(d, 3 * d, bias=False)
        self.proj = nn.Linear(d, d)
        self.drop = nn.Dropout(0.1)
        self.register_buffer('mask', torch.tril(torch.ones(block_size, block_size)).bool())
```

```
    def forward(self, x):
        B, T, _ = x.shape
        q, k, v = self.qkv(x).chunk(3, dim=-1)
        q = q.view(B, T, self.n_head, -1).transpose(1, 2)
        k = k.view(B, T, self.n_head, -1).transpose(1, 2)
        v = v.view(B, T, self.n_head, -1).transpose(1, 2)
        s = q @ k.transpose(-2, -1) / math.sqrt(k.size(-1))
        s = s.masked_fill(~self.mask[:T, :T], float('-inf'))
        z = self.drop(F.softmax(s, dim=-1)) @ v
        z = z.transpose(1, 2).contiguous().view(B, T, self.d)
        return self.drop(self.proj(z))


class Block(nn.Module):
    def __init__(self):
        super().__init__()
        self.n1, self.n2 = nn.LayerNorm(d), nn.LayerNorm(d)
        self.attn = SelfAttention()
        self.mlp = nn.Sequential(nn.Linear(d, 4 * d), nn.GELU(), nn.Linear(4 * d, d), nn.Dr

    def forward(self, x):
        x = x + self.attn(self.n1(x))
        return x + self.mlp(self.n2(x))


class GPT(nn.Module):
    def __init__(self):
        super().__init__()
        self.tok = nn.Embedding(V, d)
        self.pos = nn.Embedding(block_size, d)
        self.blocks = nn.Sequential(*[Block() for _ in range(n_layer)])
        self.norm = nn.LayerNorm(d)
        self.head = nn.Linear(d, V, bias=False)
        self.head.weight = self.tok.weight          # tied weights
        self.apply(self.init_weights)
```

```
    @staticmethod
    def init_weights(m):
        # Default embeddings have std 1. Tied to the head, that makes the first loss about
        if isinstance(m, (nn.Linear, nn.Embedding)):
            nn.init.normal_(m.weight, std=0.02)
        if isinstance(m, nn.Linear) and m.bias is not None:
            nn.init.zeros_(m.bias)

    def forward(self, idx, targets=None):
        T = idx.size(1)
        x = self.tok(idx) + self.pos(torch.arange(T, device=idx.device))
        logits = self.head(self.norm(self.blocks(x)))
        if targets is None:
            return logits, None
        return logits, F.cross_entropy(logits.view(-1, V), targets.view(-1))

    @torch.no_grad()
    def generate(self, idx, steps, T=1.0, top_k=None):
        for _ in range(steps):
            logits, _ = self(idx[:, -block_size:])
            logits = logits[:, -1] / T
            if top_k:
                kth = torch.topk(logits, min(top_k, V)).values[:, -1, None]
                logits[logits < kth] = float('-inf')
            next_id = torch.multinomial(F.softmax(logits, -1), 1)
            idx = torch.cat([idx, next_id], dim=1)
        return idx


model = GPT().to(device)
print(sum(p.numel() for p in model.parameters()), 'parameters')

# Weight decay on matrices only, not on biases or normalisation.
decay = [p for p in model.parameters() if p.ndim >= 2]
no_decay = [p for p in model.parameters() if p.ndim < 2]
opt = torch.optim.AdamW([{'params': decay, 'weight_decay': 0.01},
```

```
                        {'params': no_decay, 'weight_decay': 0.0}], lr=3e-4)


def lr_at(step, warmup=200, total=5000, base=3e-4):
    if step < warmup:
        return base * (step + 1) / warmup
    return base * 0.5 * (1 + math.cos(math.pi * (step - warmup) / (total - warmup)))


@torch.no_grad()
def estimate():
    model.eval()
    out = {}
    for split in ('train', 'val'):
        losses = [model(*get_batch(split))[1].item() for _ in range(50)]
        out[split] = sum(losses) / len(losses)
    model.train()
    return out


for step in range(5000):
    for group in opt.param_groups:
        group['lr'] = lr_at(step)
    _, loss = model(*get_batch('train'))
    opt.zero_grad(set_to_none=True)
    loss.backward()
    nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    opt.step()
    if step % 500 == 0:
        print(step, estimate())

start = torch.zeros((1, 1), dtype=torch.long, device=device)
print(''.join(itos[i] for i in model.generate(start, 400, T=0.8, top_k=40)[0].tolist()))
```

B3 LoRA on one layer

```
class LoRA(nn.Module):
    def __init__(self, layer, r=8, alpha=8.0):
        super().__init__()
        self.layer = layer
        for p in layer.parameters():
            p.requires_grad = False
        m, n = layer.weight.shape
        self.A = nn.Parameter(torch.zeros(r, n))
        self.B = nn.Parameter(torch.zeros(m, r))
        nn.init.normal_(self.A, std=1e-3)
        self.scale = alpha / r

    def forward(self, x):
        return self.layer(x) + self.scale * (x @ self.A.T @ self.B.T)
```

B4 A minimal retrieval pipeline

```
import numpy as np

def retrieve(query_vec, doc_vecs, k=3):
    q = query_vec / np.linalg.norm(query_vec)
    d = doc_vecs / np.linalg.norm(doc_vecs, axis=1, keepdims=True)
    return np.argsort(-(d @ q))[:k]

def build_prompt(question, passages):
    context = '\n\n'.join(f'[{i + 1}] {p}' for i, p in enumerate(passages))
    return ('Answer using only the passages below. Cite them by number. '
            'If they do not contain the answer, say so.\n\n'
            f'{context}\n\nQuestion: {question}\nAnswer:')
```

B5 Recall at k

```
def recall_at_k(retrieved, relevant, k):
    return len(set(retrieved[:k]) & set(relevant)) / len(relevant)
```

Part C Practice

P1. A vocabulary has 1000 tokens. What is the loss of an untrained model?

$\ln 1000 = 6.91.$

P2. Sequences of length 128 with a batch of 64. What is the shape of the logits with vocabulary V?

$64 \times 128 \times V.$

P3. $m = n = 2000$ and $r = 4$. How many parameters does LoRA train?

$2 \times 2000 \times 4 = 16{,}000$, against $4{,}000{,}000$.

P4. Why does a key-value cache speed up generation?

Each new token attends to all previous ones, but the earlier keys and values have not changed, so they need not be recomputed.

P5. A retrieval system finds 2 of the 4 relevant passages in its top 5. Find the recall at 5.

$0.5.$

P6. Why might a model miscount the letters in a word?

It sees tokens, which are chunks of characters, rather than the letters themselves.